



Formal Verification of Multi-Agent Systems

Vadim Malvone

vadim.malvone@telecom-paris.fr



(these slides are inspired by tutorials given by Aniello Murano and Wojtek Jamroga)



Outline

- 1. Introduction**
- 2. Formal Modelling**
- 3. Formal Specification of Temporal Properties**
- 4. Formal Specification of Strategic Properties**

Introduction



System Correctness

- **Hardware and software systems are growing in their abilities and applications.**
- **From health-care and transportation to smartphones, systems are becoming a fundamental tool in everyday life!**
- **System failure can effect safety and induces a loss of money, as well as time and market reputation.**



Critical systems

- **Systems in which the failure is not an option.**
- **The main critical systems are divided into:**
 - **safety, in which failure can cost lives;**
 - **mission, in which failure can cost in terms of objectives;**
 - **business, in which failure can cost money.**

Example: Therac-25

- The Therac-25 was a computer-controlled radiation therapy machine produced in 1982 after the Therac-6 and Therac-20 units.
- Problem: radiation overdose.
- Where: Canada and the USA.
- When: between 1985 and 1987.
- Consequence: three people dead and three seriously injured (of which one paralyzed).
- Why: errors in the software system and software/hardware interface (erroneous integration of pre-existing software components in the Therac-20).

Example: Pentium

- **Problem:** Prof. Thomas Nicely of Lynchburg College in Virginia found out that the Pentium chip was giving incorrect answers for some complex equations.
- **When:** 1994.
- **Consequence:** Intel was forced to withdraw the chip and replace it. All this cost Intel 450 million dollars.
- **Why:** the error was due to a design flaw of the floating point division algorithm.
- **Result:** since 1994 Intel has been using verification techniques!

How to handle system correctness?

- **Verifying the correctness of a software system before its use is a fundamental aspect.**
- **The main software verification methods are:**
 - **testing;**
 - **simulation;**
 - **formal verification.**

Testing

- **Software testing is an operation performed during the development phase.**
- **The product is checked for any malfunctions.**
- **In case of malfunctions, they are fixed before the release.**
- **Drawbacks:**
 - **not all input configurations can be checked;**
 - **input sets must be automatically generated;**
 - **no guarantee that “bad” inputs will ever be checked;**
 - **very difficult especially for multi-component systems.**

Simulation

- **Simulation is similar to testing.**
- **The verification process is performed on an abstract model of the system, by providing:**
 - input data;
 - external events.
- **Drawbacks:**
 - there is no guarantee that all possible executions will be simulated;
 - it is often several times slower than the real system;
 - it can be very expensive.



Testing & Simulation

Testing and simulation can reveal the **presence** of errors but can never establish the **absence** of errors.

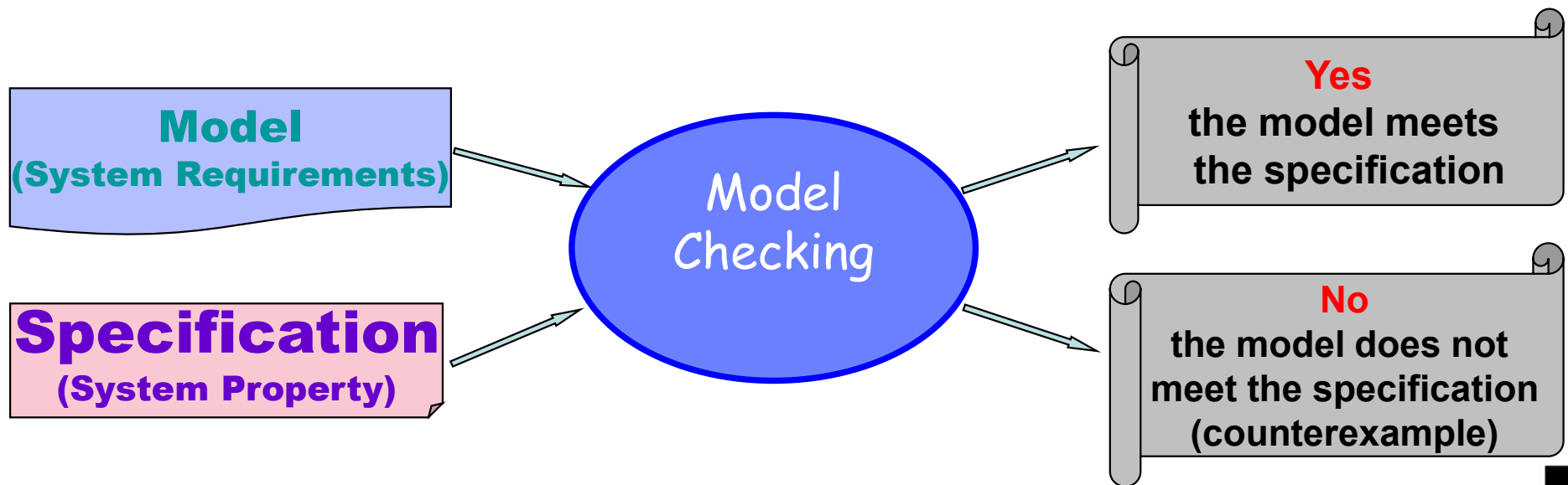


Formal Verification

- **The ability to describe systems and express specifications in a precise and unambiguous manner.**
- **The ability to clearly define when an implementation meets specifications.**
- **The properties are checked for all possible system behaviors.**
- **The analysis is exhaustive even if performed on a formal model of the system, not on the real system.**
- **One of the main approaches: Model Checking.**

Model Checking

We can check whether a system is correct with respect to a desired behavior (specification), by checking whether a formal representation of the system meets a formal specification.





Formal Modelling



System

- **Closed systems:**
 - Behavior is fully characterized by system states.
- **Open systems (system vs. environment):**
 - Interaction with an unpredictable environment.
- **Multi-agent systems (MAS):**
 - The system is composed of several entities acting adversarial or in a cooperative way.

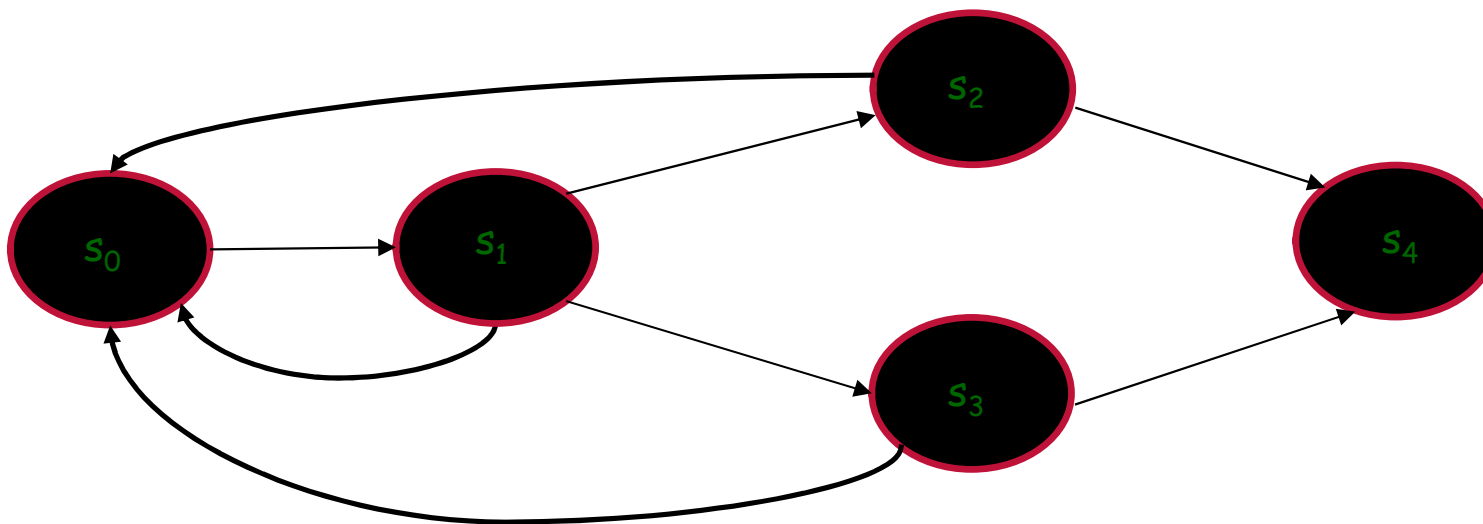
Model for Closed Systems (I)

A Kripke structure is a tuple:

$$K = (AP, S, S_0, R, Lab)$$

- AP is a set of atomic propositions.
- S is a finite set of states.
- $S_0 \subseteq S$ is the set of initial states.
- $R \subseteq S \times S$ is a **total** transition relation, that is: $\forall s \in S, \exists s': R(s, s')$.
- $Lab : S \rightarrow 2^{AP}$ labels each state with propositions true in the state.

Model for Closed Systems (II)

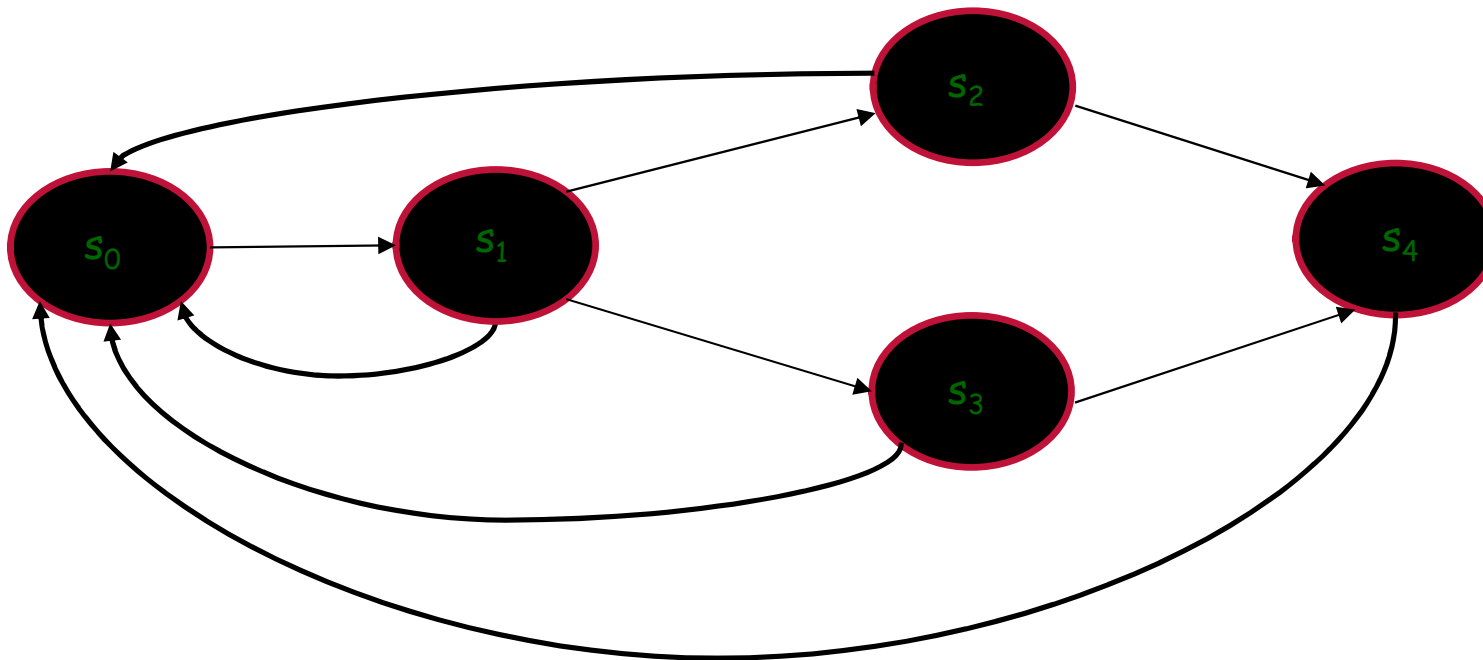


Is it a Kripke structure?

NO!

Model for Closed Systems (II)

A *path* is an infinite sequence of states $\pi = s_0, s_1, s_2, s_3, \dots$ such that for each $i \geq 0$, $R(s_i, s_{i+1})$ holds.



Is it a Kripke structure?

YES!

Is $\pi = s_0, s_1, s_4$ a path?

NO!

Is $\pi = s_0, s_1, s_2, s_4$ a path?

NO!

Is $\pi = (s_0, s_1, s_2, s_4)^\infty$ a path?

YES!

Model for MAS (I)

- It depends on the interaction between the agents.
- We can distinguish between two classes of interactions:
 - Turn-based: the states of the model are partitioned between agents and, by consequence, the owner of a state determines the next state of the model.
 - Concurrent: the agents choose actions simultaneously and independently, and the choices determine the next state of the model.

Model for MAS (II)

A Turn-based Game Structures (TGS) is a tuple:

$$TB = (AP, Ag, St, St_0, Tr, Lab)$$

- AP is a set of atomic propositions.
- $Ag = \{1, \dots, |Ag|\}$ is a set of agents.
- $St = St_1 \cup \dots \cup St_{|Ag|}$ is a set of states and $|St| = k \cdot |Ag|$, for some k .
- $St_0 \subseteq St$ is a set of initial states.
- $Tr = \{Tr_1, \dots, Tr_{|Ag|}\}$ is a set of transition functions, such that for each $i \in Ag$, $Tr_i : St_i \rightarrow 2^{St_e}$, where $e = (i \bmod |Ag|)+1$.
- $Lab : St \rightarrow 2^{AP}$ labels each state with propositions true in the state.

Model for MAS (III)

A Concurrent Game Structures (CGS) is a tuple:

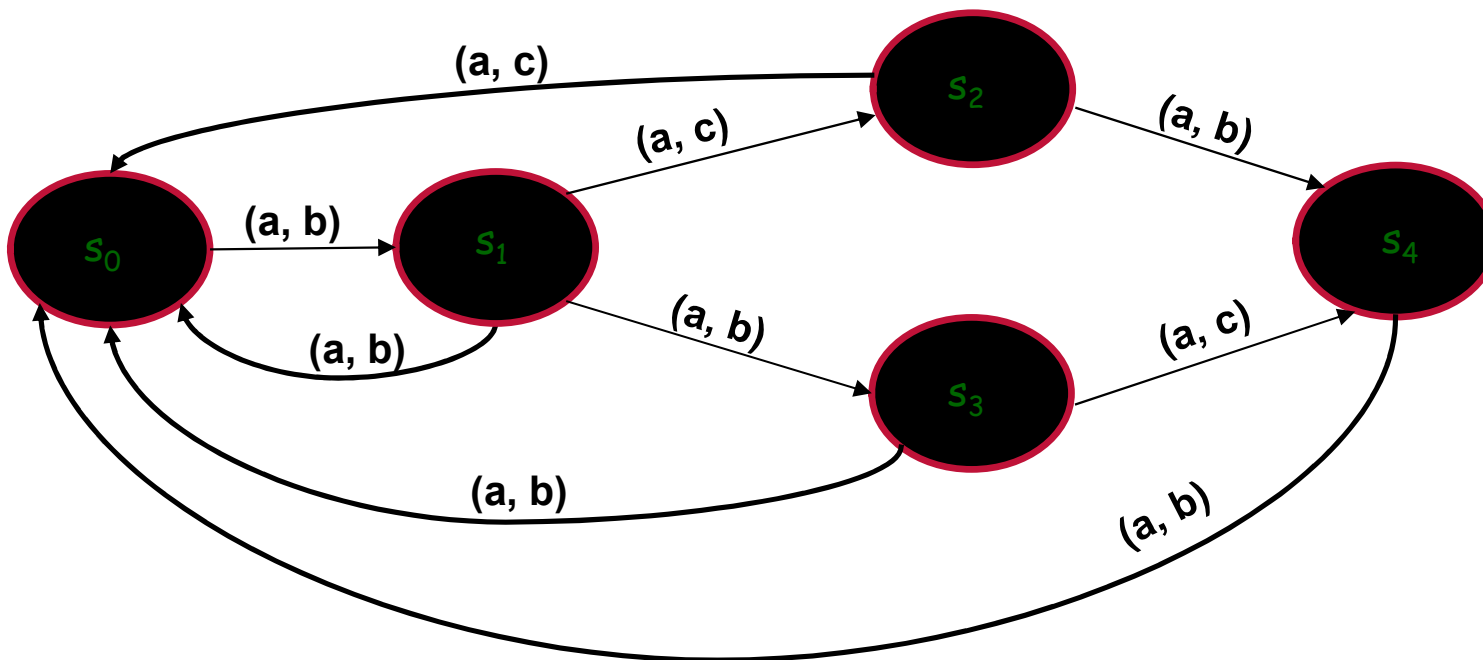
$$M = (AP, Ag, St, St_0, Ac, D, T, Lab)$$

- AP is a set of atomic propositions.
- $Ag = \{1, \dots, |Ag|\}$ is a set of agents.
- St is a set of states.
- $St_0 \subseteq St$ is a set of initial states.
- $Ac = Ac_1 \cup \dots \cup Ac_{|Ag|}$ is a set of actions.
- $D: Ag \times St \rightarrow 2^{Ac}$ defines the actions available to an agent in a state.
- $T: St \times Ac_1 \times \dots \times Ac_{|Ag|} \rightarrow St$ is a transition function.
- $Lab: St \rightarrow 2^{AP}$ labels each state with propositions true in the state.
-

Model for MAS (IV)

- A TGS $TB = (AP, Ag, St, St_0, Tr, Lab)$ can be expressed in terms of a CGS $M = (AP, Ag, St, St_0, Ac, D, T, Lab)$, as follows:
 - AP, Ag, St, St_0 , and Lab are the same elements.
 - We assign for each agent the idle action, i.e. $\forall i \in Ag, idle \in Ac_i$.
 - $\forall i \in Ag$ and $\forall s, t \in St$, if $Tr_i(s) = t$ holds then we create an action st and add it in Ac_i and $D(i, s)$. Furthermore, $\forall j \neq i, D(j, s) = \{idle\}$.
 - $T : St \times Ac_1 \times \dots \times Ac_{|Ag|} \rightarrow St$ is the transition function.
That is, given a state $s \in St_i$ there exists a tuple of actions $a_1, \dots, a_n$ such that $T(s, a_1, \dots, a_n) = t$ holds for some $t \in St$, where $a_i = st$ and $\forall j \neq i, a_j = idle$.
- So, we can focus our attention to CGSs.

Model for MAS (V)

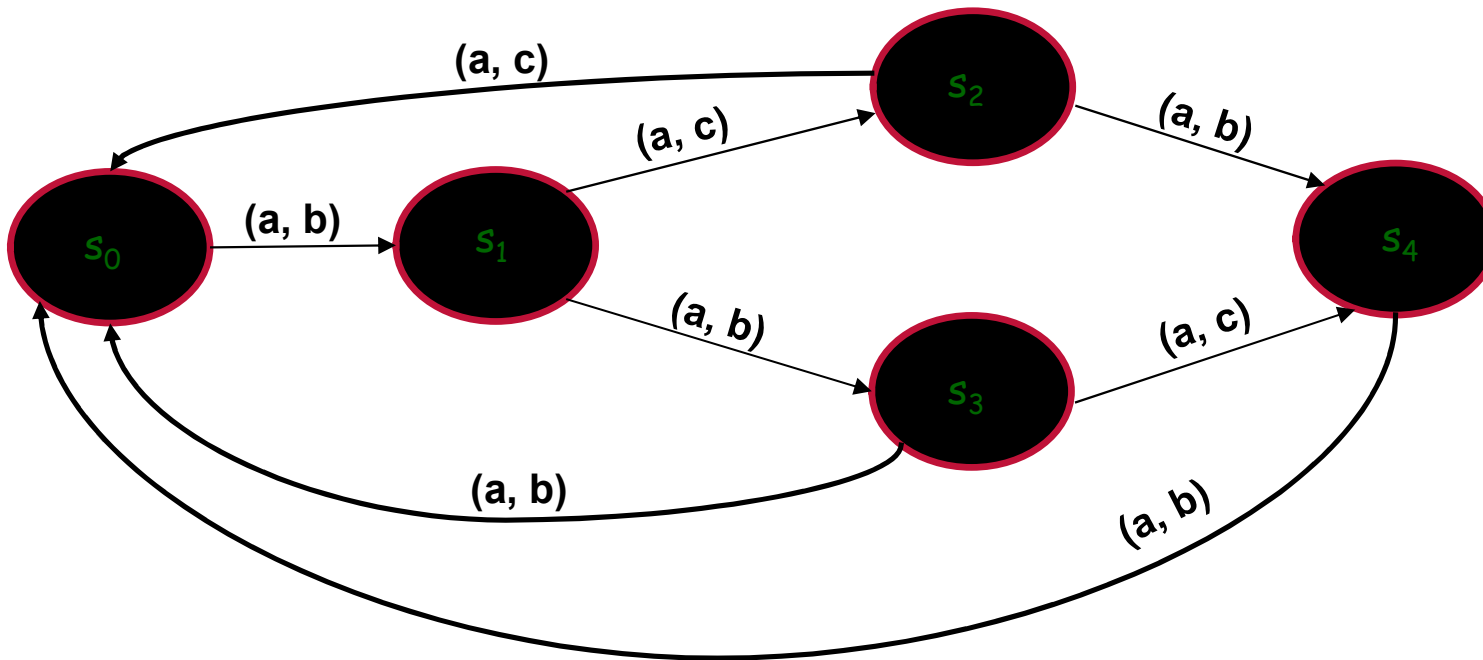


Given $Ag = \{\alpha, \beta\}$, $Ac_\alpha = \{a\}$,
 $Ac_\beta = \{b, c\}$, is it a CGS?

NO!

Model for MAS (V)

A *play* is an infinite sequence of states $\rho = s_0, s_1, s_2, s_3, \dots$ such that for each $i \geq 0$, there exists a tuple of actions $a_1, \dots, a_n$ such that $T(s_i, a_1, \dots, a_n) = s_{i+1}$ holds and $a_j \in D(j, s_i)$ for each $1 \leq j \leq n$.



Given $Ag = \{\alpha, \beta\}$, $Ac_\alpha = \{a\}$, $Ac_\beta = \{b, c\}$, is it a CGS?

YES!

Is $\rho = s_2, s_1, s_0$ a play?

NO!

Is $\rho = s_2, s_0, s_1$ a play?

NO!

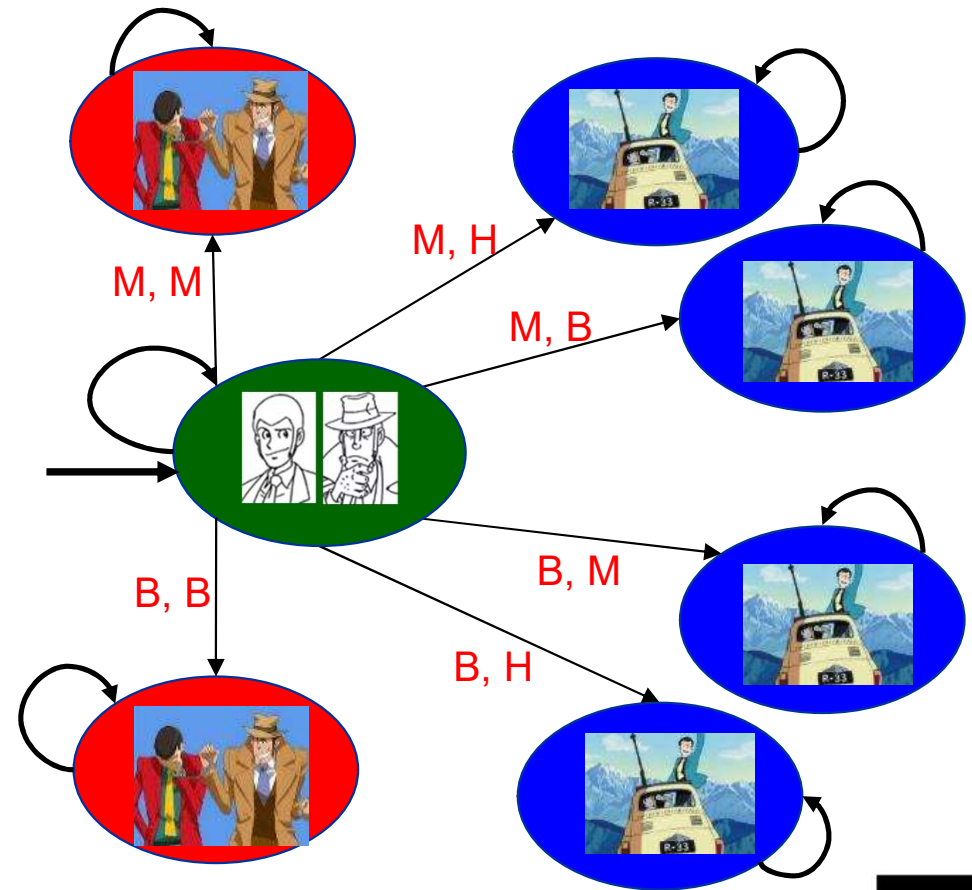
Is $\rho = (s_2, s_0, s_1)^\infty$ a play?

YES!

Example of CGS

Let $M = (AP, Ag, St, St_0, Ac, D, T, Lab)$ representing a city with a Bank and a Museum in which Lupin and Zenigata compete (Lupin starts from Home):

- $Ag = \{L, Z\}$
- $Ac = \{B, M, H\}$
- $St = \{((L, ac_1), (Z, ac_2)) \mid ac_1 \text{ and } ac_2 \text{ in } Ac\}$
- $St_0 = \{((L, H), (Z, H)), ((L, H), (Z, B)), ((L, H), (Z, M))\}$
- $AP = \{Idle, Win_L, Win_Z\}$
- Lab:
 - Lupin wins $\rightarrow Win_L$ (blue states)
 - Lupin loses $\rightarrow Win_Z$ (red states)
 - $St_0 \rightarrow Idle$ (green state)
- D and T are as in the figure





Formal Specification of Temporal Properties

Specification (I)

- Temporal logics allows to describe the dynamic behavioral properties of systems, i.e., the evolution of systems computations along the time.
- We intrinsically assume that systems computations are infinite.
- Temporal logics extend classical propositional logic with temporal operators.
- Primitive operators allow the expression of notions such as "always", "as long as", ...

Specification (II)

Depending on the underlying nature of the time, we distinguish between:

- **Linear-time temporal-logics**
 - Every moment has a unique successor
 - Infinite sequences (words)
- **Branching-time temporal-logics**
 - Every moment has several successors
 - Infinite trees

LTL: Linear-time Temporal Logic

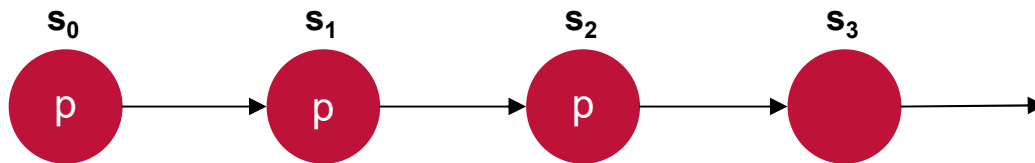
- It determines patterns on infinite traces.
- At any time, there can be only one future.
- Elements:
 - Atomic propositions: AP
 - Boolean operators: $\neg$ (NOT), $\vee$ (OR), $\wedge$ (AND)
 - Temporal operators:
 - X (NEXT)
 - F (EVENTUALLY)
 - G (GLOBALLY)
 - U (UNTIL)

Additional operators: Weak Until (I)

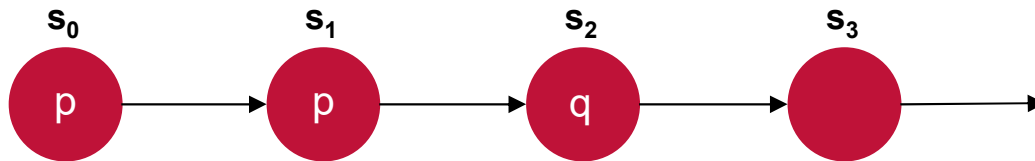
- It is the Until in which the stop condition is not required to occur anymore.
- Thus, $\psi_1 W \psi_2$ is true if one between $\psi_1 U \psi_2$ and $G \psi_1$ holds.
- Formal semantics:
 - $\pi \models \psi_1 W \psi_2$ iff $\forall i \geq 1, \pi_{\geq i} \models \psi_1$ or
 $\exists i \geq 0$, such that $\pi_{\geq i} \models \psi_2$ and
 $\forall 0 \leq j < i$, we have that $\pi_{\geq j} \models \psi_1$
- Also for W it holds that $\psi_1 W \psi_2 = \psi_2 \vee (\psi_1 \wedge X(\psi_1 W \psi_2))$.

Additional operators: Weak Until (II)

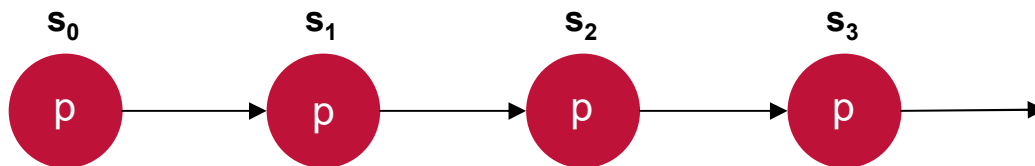
$p \text{ W } q$



✗



✓



✓

Additional operators: Release (I)

■ $\psi_1 R \psi_2$:

- ψ_2 is true until and including once ψ_1 becomes true, or
- ψ_2 remains true forever.

■ Formal semantics:

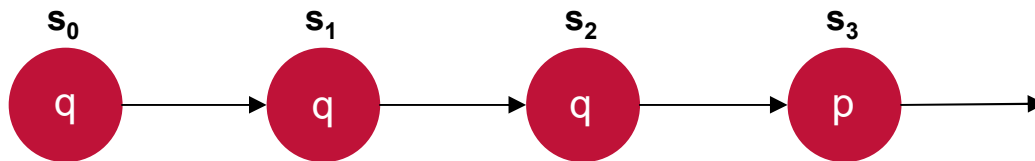
- $\pi \models \psi_1 R \psi_2$ iff $\forall i \geq 0, \pi_{\geq i} \models \psi_2$ or $\exists i \geq 0$ such that $\pi_{\geq i} \models \psi_1$ and
 $\forall 0 \leq j \leq i, \pi_{\geq j} \models \psi_2$

■ The Release is the dual of the Until.

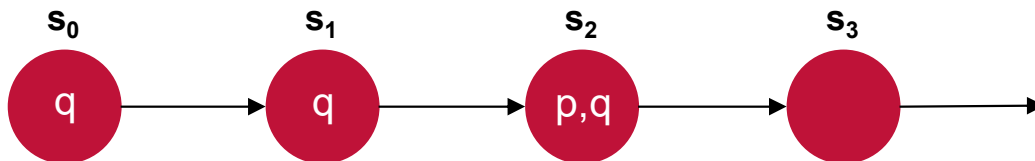
■ Some abbreviations: $\psi_1 R \psi_2 = \neg (\neg \psi_1 U \neg \psi_2) = \psi_2 \wedge (\psi_1 \vee X(\psi_1 R \psi_2))$.

Additional operators: Release (II)

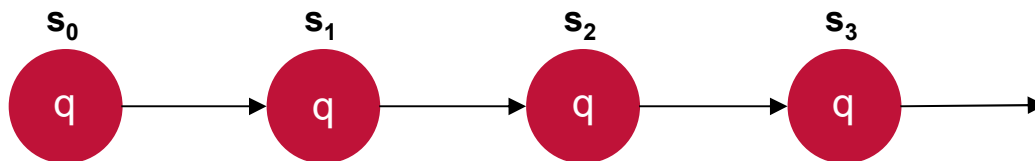
p R q



x



✓



✓

LTL Model Checking

Given a Kripke structure $K = (AP, S, S_0, R, Lab)$ modelling the system and an LTL formula ψ over AP representing the specification.

The LTL model checking problem is to check whether

$$\pi \models \psi$$

for each path π starting in $s_0 \in S_0$

Branching-time Temporal Logics

- An LTL formula is satisfied over a system model K if it is satisfied on every path starting at the initial states.
- Paths in K represent all possible system computations.
- To check a formula on some paths of K , we need a logic that allows us to talk about branches of K .
- We study CTL and CTL*.

Computation Tree Logic (CTL)

- CTL uses the same temporal operators as LTL.
- Additionally, we use two path quantifiers:
 - **E** means ‘for some paths’
 - **A** means ‘for all paths’

EX, EF, EG, EU

AX, AF, AG, AU

- In CTL, formulas are investigated on states rather than paths.

CTL Model Checking

Given a Kripke structure $K = (AP, S, S_0, R, Lab)$ modelling the system and a CTL formula φ over AP representing the specification.

The CTL model checking problem is to check whether

$$s \models \varphi$$

for each $s \in S_0$

Branching vs Linear

- LTL makes it possible to state the properties more simply.
- CTL is preferable to avoid combinatorial explosion problems during exhaustive verification.
- LTL and CTL can be seen as fragments of the same logic: CTL*.
- CTL* allows complex nesting such as:

AXX, EFG, AGX, AFG, EXFG, ...

Formal Syntax of CTL*

- CTL* state-formulas are formed according to the grammar:

$$\varphi := p \mid \neg\varphi \mid \varphi \vee \varphi \mid E \psi$$

where $p \in AP$ and ψ is a path-formula

- CTL* path-formulas are formed according to the grammar:

$$\psi := \varphi \mid \neg\psi \mid \psi \vee \psi \mid X \psi \mid \psi U \psi$$

where φ is a state-formula

Syntax of LTL in terms of state and path formulas

- LTL state-formulas are formed according to the grammar:

$$\varphi := A \psi$$

where $p \in AP$ and ψ is a path-formula

- LTL path-formulas are formed according to the grammar:

$$\psi := p \mid \neg\psi \mid \psi \vee \psi \mid X\psi \mid \psi U \psi$$

Syntax of CTL in terms of state and path formulas

- CTL state-formulas are formed according to the grammar:

$$\varphi := p \mid \neg\varphi \mid \varphi \vee \varphi \mid E\psi \mid A\psi$$

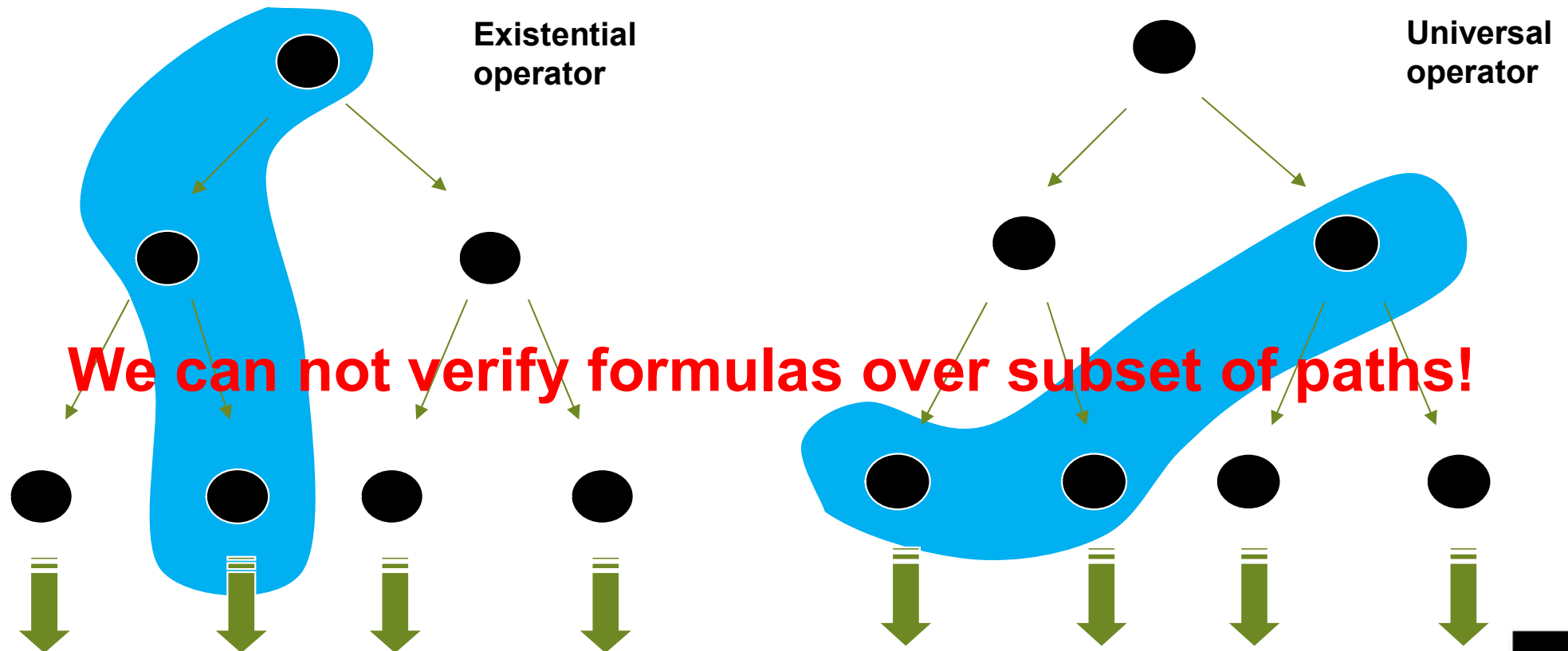
where $p \in AP$ and ψ is a path-formula

- CTL path-formulas are formed according to the grammar:

$$\psi := X\varphi \mid \varphi U \varphi$$

where φ is a state-formula

What we can do with CTL and CTL*





Formal Specification of Strategic Properties

Alternating-Time Temporal Logic

- ATL allows to specify complex interactions in a multi-agent setting.
- In ATL, the temporal logic framework fully meets game theory.
- ATL is a generalization of CTL, in which temporal operators are indexed by coalitions of agents.
- Formally, path quantifiers E and A are replaced with strategic cooperative quantifiers of the form $\langle\langle C \rangle\rangle$ and $[[C]]$, where C is a coalition of agents.

Syntax of ATL (I)

- ATL state-formulas are formed according to the grammar:

$$\varphi := p \mid \neg\varphi \mid \varphi \vee \varphi \mid \langle\langle A \rangle\rangle \psi \mid [[A]] \psi$$

where $p \in AP$ and ψ is a path-formula

- In ATL path-formulas are as in CTL:

$$\psi := X \varphi \mid \varphi U \varphi$$

where φ is a state-formula

Syntax of ATL (II)

■ Derived state operators

- $\text{true} = p \vee \neg p$
- $\text{false} = p \wedge \neg p$
- $\varphi \wedge \psi = (\neg \varphi) \vee (\neg \psi)$

■ Derived path operators

- $F \varphi = \text{true} \mathbf{U} \varphi$
- $G \varphi = \neg F \neg \varphi$

Strategies (I)

- It is a conditional plan that specifies which decision the agent α has to take in every possible situation.
- A **memoryfull** strategy for an agent α is a function

$$\sigma_{\alpha}: \text{St}^+ \rightarrow \text{Ac}$$

- We can have a **memoryless** strategy via the function

$$\sigma_{\alpha}: \text{St} \rightarrow \text{Ac}$$

Strategies (II)

- A collective strategy Σ_A for a group of agents A is a tuple of strategies, one for each agent in A , i.e. $\Sigma_A = \{\sigma_1, \dots, \sigma_{|A|}\}$.
- The outcome of the team A from a state s , $\text{out}(s, \Sigma_A)$, is the set of all paths that result from agents A executing Σ_A .

Are you able to formally define $\text{out}(s, \Sigma_A)$?

$$\text{out}(s, \Sigma_A) = \{\pi \in S^\infty \mid \forall i \geq 0, T(\pi_i, a_1, \dots, a_n) = \pi_{i+1} \text{ and } \dots \\ \forall j \in A, \sigma_j(\pi_i) = a_j \text{ and } \forall k \in Ag \setminus A, a_k \in D(\pi_i, k)\}$$

where π_i is i -th element of π , with $i \geq 0$.

Informal Semantics of ATL

- **$\langle\langle A \rangle\rangle \varphi$ means that coalition A has a collective strategy to enforce φ , no matter what the other agents (not in A) will do.**
- **Strategic quantifiers allow for a selective extraction of paths over a game model, i.e. subset of paths!**

Formal Semantics of ATL (I)

- Given a CGS $M = (AP, Ag, St, St_0, Ac, D, T, Lab)$.
- Given a state $s \in St$ of M and a path $\pi = s_0 s_1 s_2 s_3 \dots$.
- $M, s \models \varphi$ means that the ATL state formula φ holds on s .
- $M, \pi \models \psi$ means that the ATL path formula ψ holds on π .
- Let π_i be the i -th element of π , with $i \geq 0$.

Formal Semantics of ATL (II)

The relation $\models$ is defined inductively as follows:

- $M, s \models p$ iff $p \in \text{Lab}(s)$
- $M, s \models \neg\varphi$ iff not $M, s \models \varphi$
- $M, s \models \varphi_1 \vee \varphi_2$ iff $M, s \models \varphi_1$ or $M, s \models \varphi_2$
- $M, s \models \langle\langle A \rangle\rangle \varphi$ iff there is Σ_A such that for every $\pi \in \text{out}(s, \Sigma_A)$, $M, \pi \models \varphi$
- $M, s \models [[A]] \varphi$ iff for all Σ_A , there is $\pi \in \text{out}(s, \Sigma_A)$ such that $M, \pi \models \varphi$
- $M, \pi \models X\varphi$ iff $M, \pi_1 \models \varphi$
- $M, \pi \models \varphi_1 \cup \varphi_2$ iff $\exists i \geq 0$, such that $M, \pi_i \models \varphi_2$ and
 $\forall 0 \leq j < i$, we have that $M, \pi_j \models \varphi_1$

ATL Model Checking

Given a CGS $M = (AP, Ag, St, St_0, Ac, D, T, Lab)$ modelling the multi-agent system and a ATL formula φ over AP representing the specification.

The ATL model checking problem is to check whether

$$s \models \varphi$$

for each $s \in St_0$

Examples of formulas

■ Agents = {  ,  ,  }

- Lupin always runs away from Zenigata and at a certain point he gets the diamonds

«Lupin» (G run away $\wedge$ F diamonds)

- Lupin and Margot have fun with money until they get caught from Zenigata

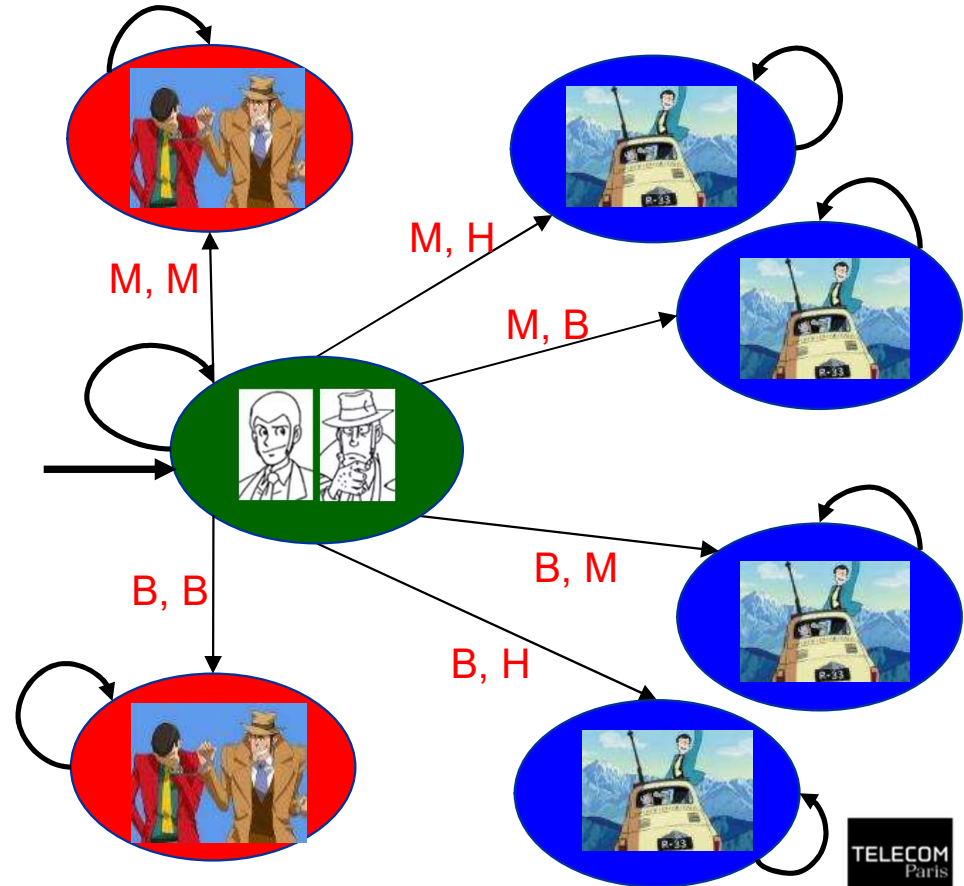
«Lupin, Margot» fun Until caught



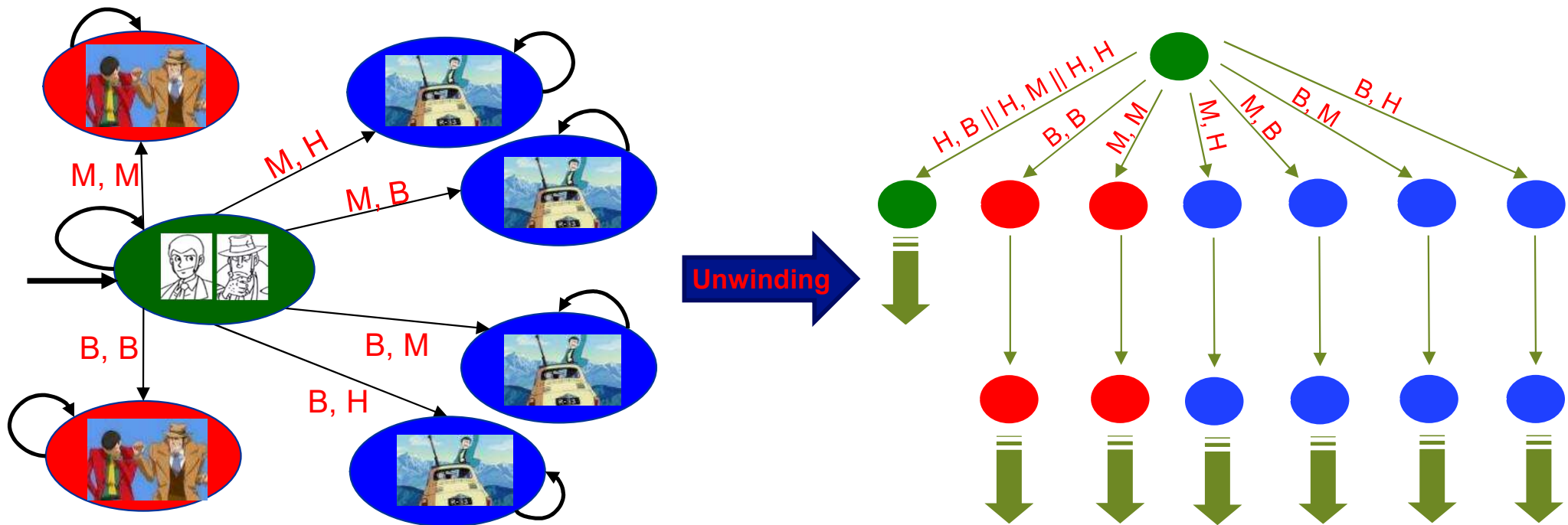
Example of formula for the Lupin-Zenigata CGS (I)

$$M \models \ll \text{Zenigata} \gg F \text{Win}_z$$

How can we verify this property?



Example of formula for the Lupin-Zenigata CGS (II)

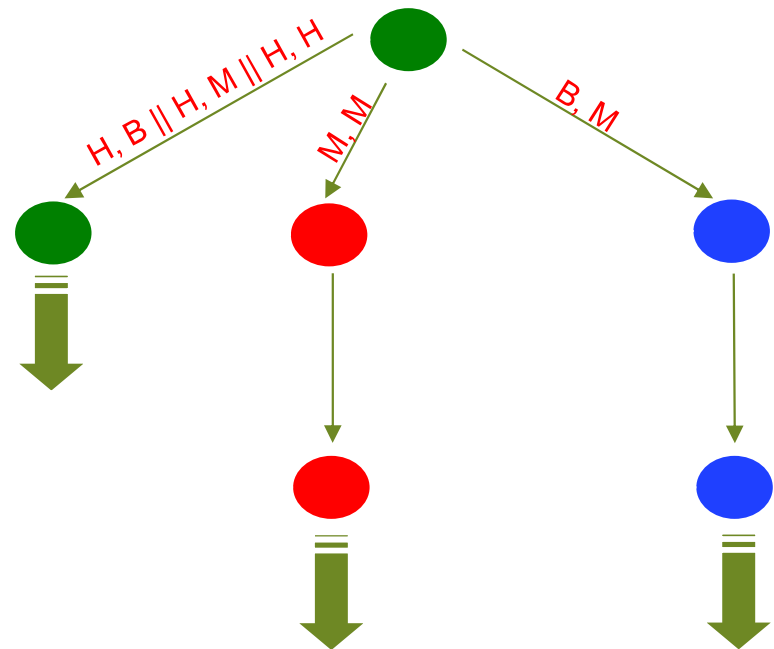


Example of formula for the Lupin-Zenigata CGS (III)

$M \models \ll \text{Zenigata} \gg F \text{Win}_z$

Select the strategy for Zenigata

- Zenigata selects M;

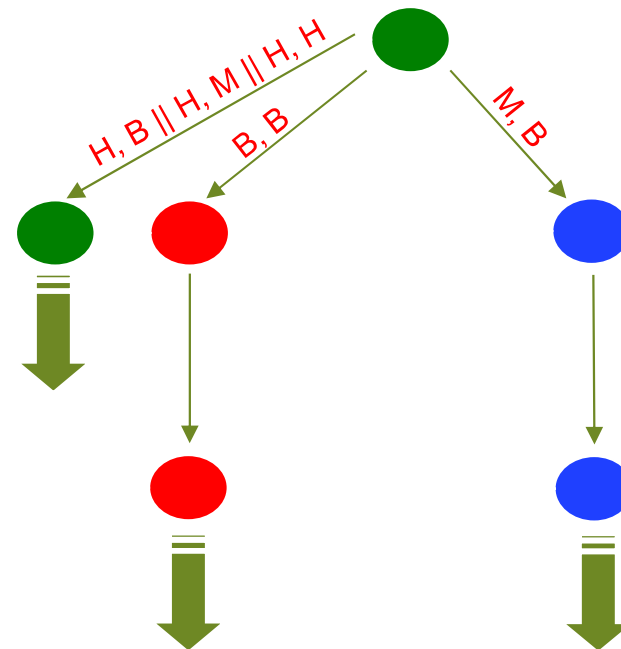


Example of formula for the Lupin-Zenigata CGS (III)

$M \models \ll \text{Zenigata} \gg F \text{Win}_z$

■ Select the strategy for Zenigata

- Zenigata selects M;
- Zenigata selects B;



Example of formula for the Lupin-Zenigata CGS (III)

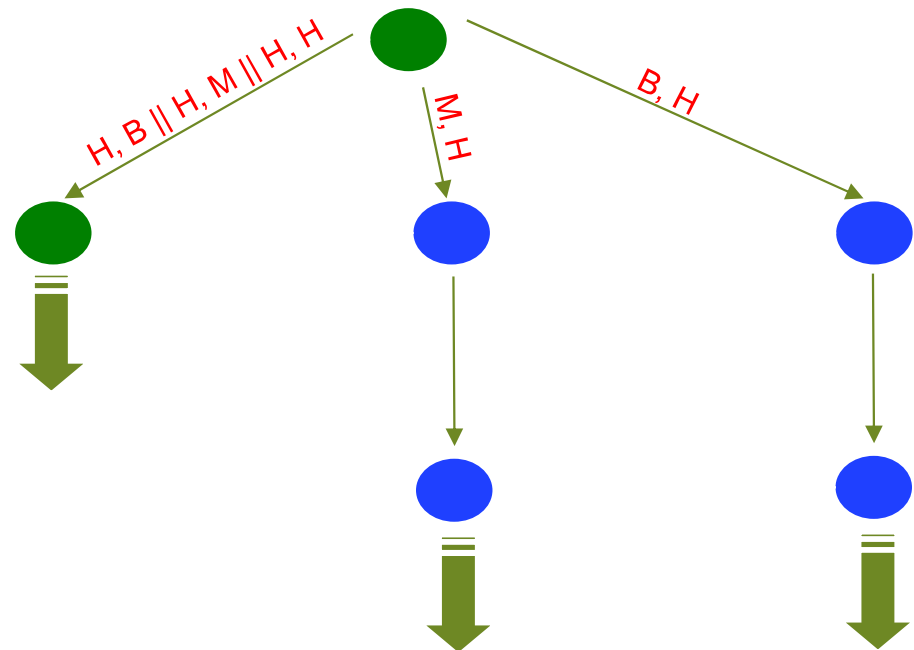
$M \models \ll \text{Zenigata} \gg F \text{Win}_z$

■ Select the strategy for Zenigata

- Zenigata selects M;
- Zenigata selects B;
- Zenigata selects H.

■ The formula is false.

■ Lupin can take actions different from those taken by Zenigata.



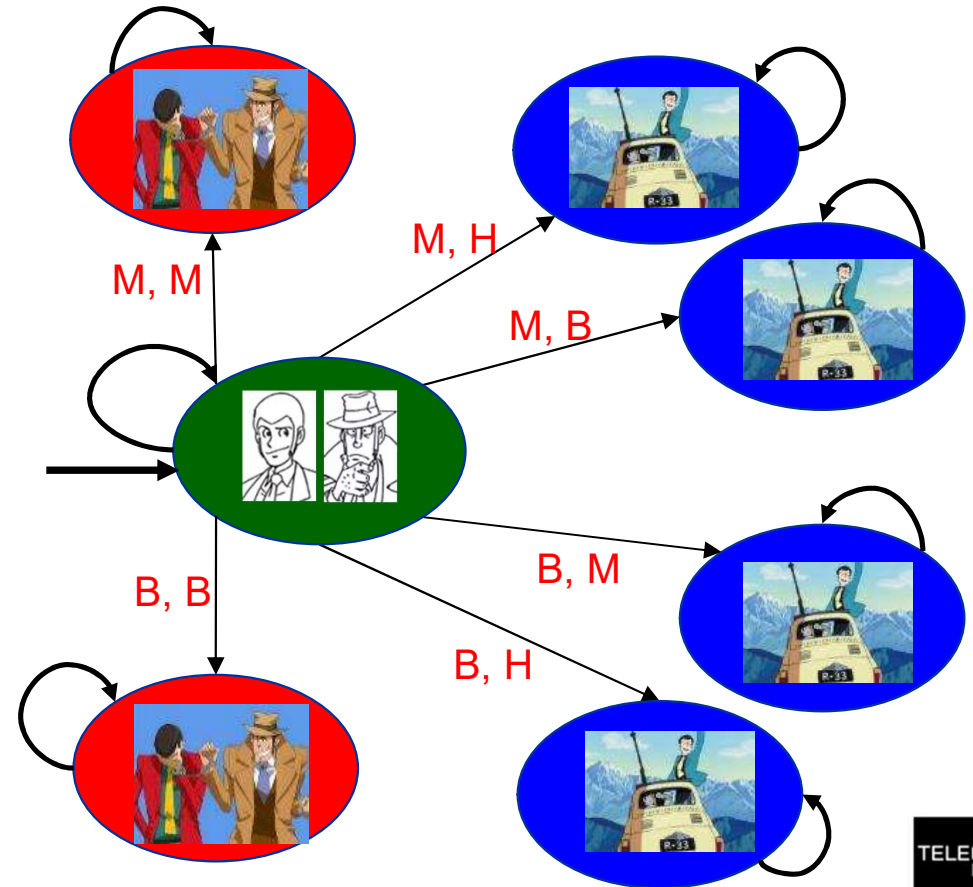
Exercise: verify the following formulas in the Lupin-Zenigata CGS

■ $M \models [[\text{Lupin}]] F \text{Win}_L$

- The formula is false.

■ $M \models \ll \text{Lupin}, \text{Zenigata} \gg F \text{Win}_L$

- The formula is true.



Semantic Embedding of CTL in ATL

Temporal reasoning can be semantically embedded in strategic reasoning as follows:

- We take a Kripke structure to be a concurrent game structure with a single agent (“the system” sys).
- Transitions are due to actions of the agent sys .
- $E \varphi$ (“there is a path on which φ holds”) can be then translated to $\langle\langle \text{sys} \rangle\rangle \varphi$ (“the system can behave in a way that makes φ true”).
- $A \varphi$ (“for all paths, φ holds”) can be translated to $\langle\langle \emptyset \rangle\rangle \varphi$ (“is enforced whatever all the agents – i.e., the system – do”).

Syntactic Embedding of CTL in ATL

Also, ATL extends CTL by the following syntactic translation:

- $A \varphi \equiv \langle\langle \emptyset \rangle\rangle \varphi$ (“for all paths” = necessary outcomes)
- $E \varphi \equiv \langle\langle Ag \rangle\rangle \varphi$ (“there is a path” = outcomes obtainable by grand coalition)

Memory Does not Influence Ability in ATL

Let us discern between two definitions of the satisfaction relation:

- $\models_R$: memoryfull is assumed, strategies are of type $\sigma : St^+ \rightarrow Ac$
- $\models_r$: only memoryless strategies are allowed, i.e., $\sigma : St \rightarrow Ac$

For any M , s , and φ , we have:

$$M, s \models_r \varphi \leftrightarrow M, s \models_R \varphi$$

To improve expressiveness, as for CTL, we have ATL*.

Syntax of ATL*

- ATL* state-formulas are formed according to the grammar:

$$\varphi := p \mid \neg\varphi \mid \varphi \vee \varphi \mid \langle\langle A \rangle\rangle \psi$$

where $p \in AP$ and ψ is a path-formula

- ATL* path-formulas are as in CTL*:

$$\psi := \varphi \mid \neg\psi \mid \psi \vee \psi \mid X\psi \mid \psi U \psi$$

where φ is a state-formula

Formal Semantics of ATL* (I)

- Given a CGS $M = (AP, Ag, St, St_0, Ac, D, T, Lab)$.
- Given a state $s \in S$ of M and a path $\pi = s_0 s_1 s_2 s_3 \dots$.
- $M, s \models \varphi$ means that the ATL state formula φ holds on s .
- $M, \pi \models \psi$ means that the ATL path formula ψ holds on π .
- Let π_i be the i -th element of π , with $i \geq 0$.
- Let $\pi_{\geq i}$ be the suffix of π starting at $i \geq 0$.

Formal Semantics of ATL* (II)

The relation $\models$ is defined inductively as follows:

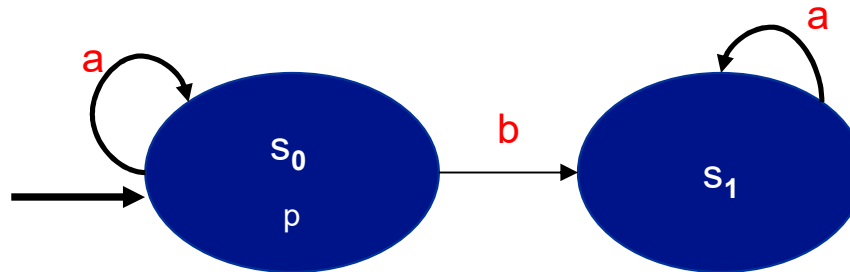
- $M, s \models p$ iff $p \in \text{Lab}(s)$
- $M, s \models \neg\varphi$ iff not $M, s \models \varphi$
- $M, s \models \varphi_1 \vee \varphi_2$ iff $M, s \models \varphi_1$ or $M, s \models \varphi_2$
- $M, s \models \langle\langle A \rangle\rangle \varphi$ iff there is Σ_A , such that for every $\pi \in \text{out}(s, \Sigma_A)$, $M, \pi \models \varphi$
- $M, \pi \models \varphi$ iff $M, \pi_0 \models \varphi$
- $M, \pi \models \neg\psi$ iff not $M, \pi \models \psi$
- $M, \pi \models \psi_1 \vee \psi_2$ iff $M, \pi \models \psi_1$ or $M, \pi \models \psi_2$
- $M, \pi \models X\psi$ iff $M, \pi_{\geq 1} \models \psi$
- $M, \pi \models \psi_1 \cup \psi_2$ iff $\exists i \geq 0$, such that $M, \pi_{\geq i} \models \psi_2$ and
 $\forall 0 \leq j < i$, we have that $M, \pi_{\geq j} \models \psi_1$

For ATL* memory matters

There is a model M , a state s , and a formula φ , such that

$$M, s \models_r \varphi \neq M, s \models_R \varphi$$

Counterexample: $\langle\langle a \rangle\rangle (X p \wedge X X \neg p)$



Connection to Games

■ $\langle\langle A \rangle\rangle \varphi$

- $\langle\langle A \rangle\rangle$ splits the agents into proponents and opponents.
- φ defines the winning condition.

infinite 2-player zero-sum game

■ Solving a game $\approx$ checking if $M, s \models \langle\langle A \rangle\rangle \varphi$

■ Model checking ATL corresponds to game solving in game theory!

Exercises

Specification of contract signing

Formalize the following statements in ATL:

1. Alice is able to sign the contract sometime in the future.

$\ll\text{Alice}\gg F \text{ sign}$

2. If Alice signs the contract then Bob can sign it as well or refuse to sign. The converse is not necessarily true.

$\ll\text{Alice}\gg F \text{ sign} \rightarrow \ll\text{Bob}\gg F (\text{sign} \vee \text{refuse})$

Specification of contract signing

Formalize the following statements in ATL*:

1. Whenever Alice receives a document, she signs it immediately.

$\ll\text{Alice}\gg F (\text{receive} \rightarrow \text{sign})$

2. If Alice and Bob can send documents to each other infinitely often, and Alice signs whatever she gets immediately, then Bob can sign infinitely many documents.

$(\ll\text{Alice}\gg GF \text{ send} \wedge \ll\text{Bob}\gg GF \text{ send} \wedge \ll\text{Alice}\gg F (\text{receive} \rightarrow \text{sign})) \rightarrow \ll\text{Bob}\gg GF \text{ sign}$